

소켓프로그래밍2

김학재

2026.05.20

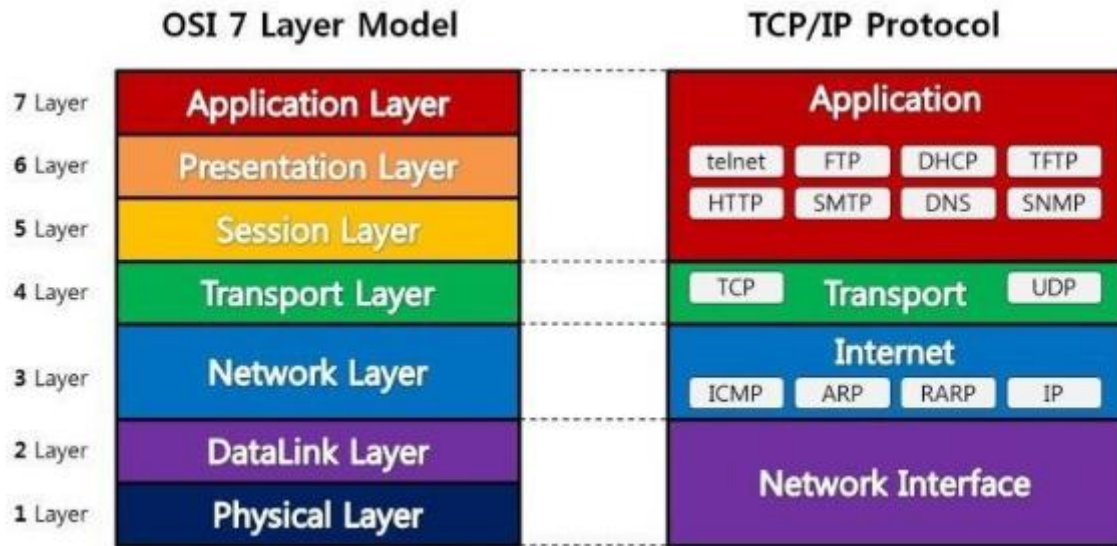
목차

1. 의문점 해소
2. 코드 설명
3. 주소체계와 데이터 정렬
4. Echo서버, 클라이언트 구현하기

OSI

OSI (Open Systems Interconnection) :

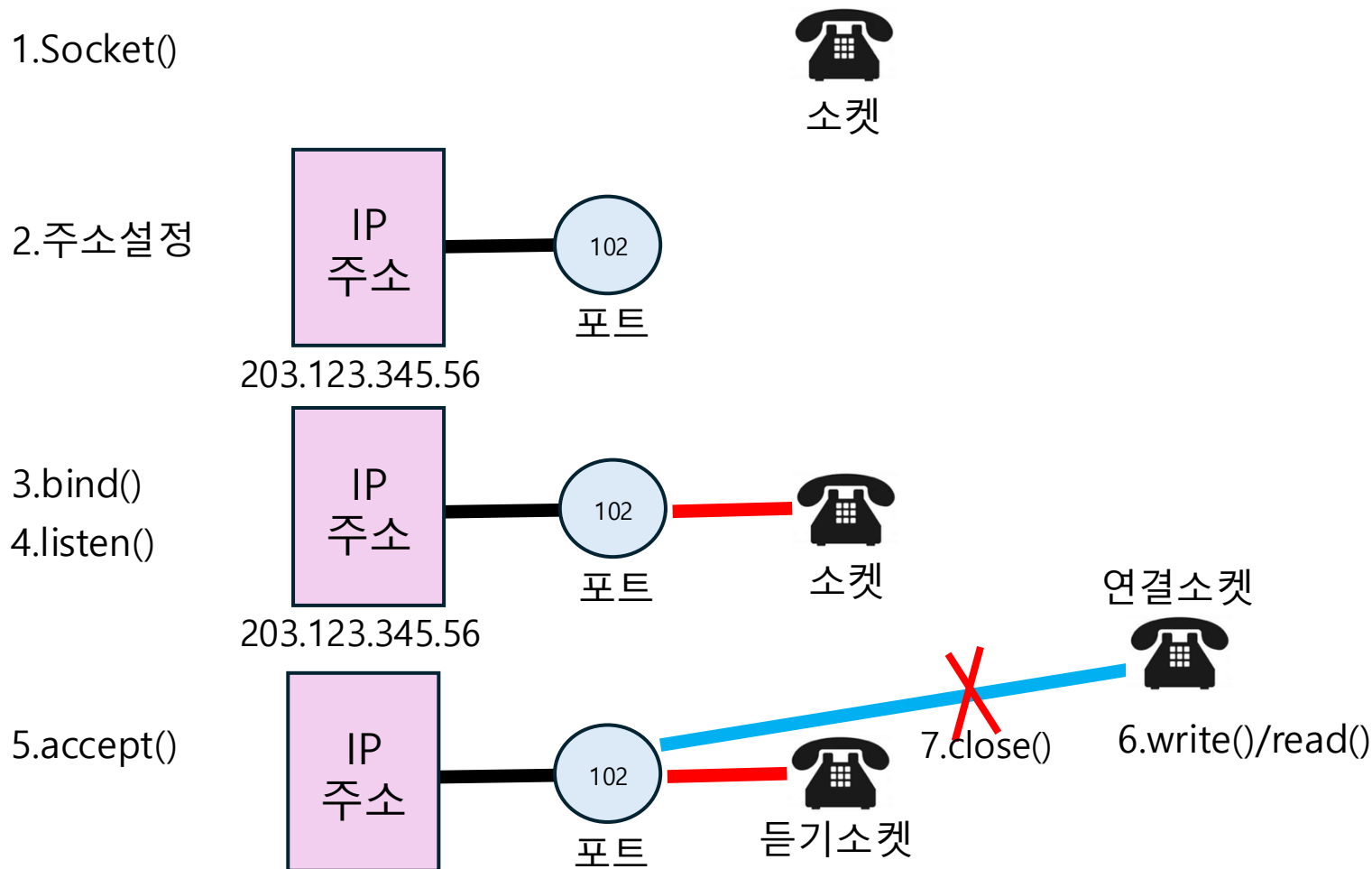
서로 다른 컴퓨터들이 표준화된 방식으로 통신할 수 있도록 만든 참조 모델.
복잡한 네트워크를 역할별로 나눠서 관리/개발/수리를 쉽게 하기 위함.



TCP/IP Protocol : 실제 인터넷 환경에서 사용되는 실용적인 프로토콜 모델로,
OSI 7계층 중 비슷한 역할을 하는 계층은 묶어서 4계층 구조로 표현한다.

서버 흐름

- 1.socket()
- 2.주소설정
- 3.bind()
- 4.listen()
- 5.accept()
- 6.write() / read()
- 7.close()



서버 코드

```
11 int main(int argc, char *argv[])
12 {
13     int serv_sock, clnt_sock; // 파일 디스크립터 번호로 관리
14     char message[BUF_SIZE];
15     int str_len, i=0;
16
17     struct sockaddr_in serv_adr;
18     struct sockaddr_in clnt_adr;
19     socklen_t clnt_adr_sz;
20
21     if(argc!=2) {
22         printf("Usage : %s <port>\n", argv[0]);
23         exit(1);
24     }
25
26     serv_sock=socket(PF_INET, SOCK_STREAM, 0);
27     if(serv_sock==-1)
28         error_handling("socket() error");
29
30     memset(&serv_adr, 0, sizeof(serv_adr));
31     serv_adr.sin_family=AF_INET;
32     serv_adr.sin_addr.s_addr=htonl(INADDR_ANY);
33     serv_adr.sin_port=htons(atoi(argv[1]));
34
```

```
35
36     if(bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr))==-1)
37         error_handling("bind() error");
38
39     if(listen(serv_sock, 5)==-1)
40         error_handling("listen() error");
41
42     clnt_adr_sz=sizeof(clnt_adr);
43
44     while(1)
45     {
46         clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);
47         if(clnt_sock==-1)
48             error_handling("accept() error");
49         else
50             printf("Connected client %d \n", ++i);
51
52         while((str_len=read(clnt_sock, message, BUF_SIZE))!=0)
53             write(clnt_sock, message, str_len);
54
55         close(clnt_sock);
56     }
57
58     close(serv_sock);
59     return 0;
60 }
```

클라에서 connect요청한 소켓들
5개까지 대기 시킬 수 있음

그럼 하나씩 빼서 accept한다

연결 과정

```
sock=socket(PF_INET, SOCK_STREAM, 0);
```

```
if(sock==-1)  
    error_handling("socket() error");
```

```
memset(&serv_adr, 0, sizeof(serv_adr));
```

```
serv_adr.sin_family=AF_INET;
```

```
serv_adr.sin_addr.s_addr=inet_addr(argv[1]);
```

```
serv_adr.sin_port=htons(atoi(argv[2]));
```

```
if(connect(sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr))==-1)
```

```
    error_handling("connect() error!");
```

```
else
```

```
    puts("Connected.....");
```

클라이언트

socket()

connect()

서버

socket()

bind()

listen()

accept()



통신소켓의 포트번호

```
clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);
if(clnt_sock==-1)
    error_handling("accept() error");
else
    printf("Connected client %d \n", ++i);
```

```
printf("serv_sock IP      : %s \n", inet_ntoa(serv_adr.sin_addr));
printf("serv_sock PORT    : %d \n", ntohs(serv_adr.sin_port));
printf("client socket IP    : %s \n", inet_ntoa(clnt_adr.sin_addr));
printf("client socket PORT  : %d \n", ntohs(clnt_adr.sin_port));
```

```
struct sockaddr_in temp;
socklen_t len = sizeof(temp);
```

```
getsockname(clnt_sock, (struct sockaddr*)&temp, &len);
```

```
printf("clnt_sock IP      : %s\n", inet_ntoa(temp.sin_addr));
printf("clnt_sock PORT    : %d\n", ntohs(temp.sin_port));
```

```
k@k:/mnt/c/Users/k/Desktop/SocketP/IterativeEcho$ ./eserver 4567 k@k:/mnt/c/Users/k/Desktop/SocketP/IterativeEcho$ ./eclient 127.0.0.1 4567
Connected client 1
serv_sock IP      : 0.0.0.0
serv_sock PORT    : 4567
client socket IP  : 127.0.0.1
client socket PORT : 47582
clnt_sock IP      : 127.0.0.1
clnt_sock PORT    : 4567
|
Connected.....
Input message(Q to quit): q
```

서버

클라이언트

코드 설명

int socket(int domain, int type, int protocol);
성공 시 파일 디스크립터 반환, 실패시 -1 반환

domain : 주소체계 (IPv4, IPv6)
type : 통신방식 (TCP, UDP)
protocol : 세부 프로토콜

```
sock=socket(PF_INET, SOCK_STREAM, 0);
```

int bind(int sockfd, struct sockaddr *myaddr, socklen_t addrlen);
성공시 0 반환, 실패시 -1 반환

sockfd : 소켓 파일디스크립터
myaddr : 주소 (ip, port)
addrlen : 주소 구조체 크기

int listen(int sockfd, int backlog);
성공시 0 반환, 실패시 -1 반환

backlog : 대기큐 길이

int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
성공시 파일 디스크립터 반환, 실패시 -1 반환

addr : 접속한 상대방 주소(ip, port)

int connect(int sockfd, struct sockaddr *serv_addr, socklen_t addrlen);
성공시 0반환, 실패시 -1 반환

addr : 연결할 서버 주소

코드 설명

소켓 만들 때 사용

Protocol Family

PF_INET : IPv4 인터넷 프로토콜 체계 (4 Byte 주소체계)

PF_INET6 : IPv6 인터넷 프로토콜 체계 (16 Byte 주소체계)

주소 구조체 만들 때 사용

Address Family

AF_INET : IPv4 인터넷 프로토콜에 적용하는 주소체계

AF_INET6 : IPv6 인터넷 프로토콜에 적용하는 주소체계

둘 다 int값이 같은데 이름으로 의미를 주는 것

구조체 설명

struct sockaddr // 모든 주소를 받기 위한 범용 구조체.

```
{
    sa_family_t    sa_family; // 주소 종류
    char          sa_data[14]; // 주소 데이터
};
```

struct sockaddr_in // IPv4전용 , sockaddr_in6 : IPv6전용

```
{
    sa_family_t    sin_family; // 주소체계 (AF_INET)
    uint16_t       sin_port;    // 16비트 TCP/UDP
    struct in_addr sin_addr;    // 32비트 IP주소
    char          sin_zero[8]; // 사용되지않음, soackaddr 크기맞추기위해 넣은 패딩
};
```

struct in_addr

```
{
    in_addr_t      s_addr; // 32비트 IPv4 인터넷 주소
}
```

```
struct sockaddr_in clnt_addr;
```

```
if(bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr))==-1)
```

```
clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_sz);
```

```
memset(&serv_addr, 0, sizeof(serv_addr));
```

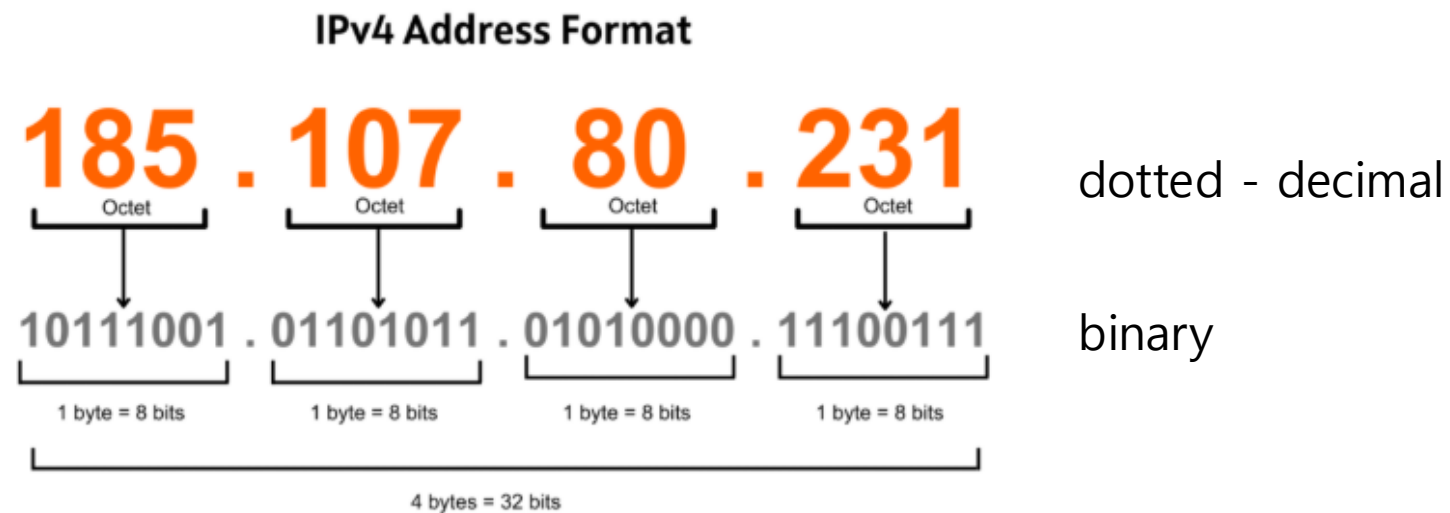
```
serv_addr.sin_family=AF_INET;
```

```
serv_addr.sin_addr.s_addr=htonl(INADDR_ANY); // 모든 인터페이스 허용 0.0.0.0
```

```
serv_addr.sin_port=htons(atoi(argv[1])); // 포트번호 들어있는 곳
```

주소체계

주소 IPv4 (4바이트), IPv6(16바이트)



IP 주소 표현방식

- 도메인 이름 문자열 "www.naver.com"
- 점과 10진수 문자열 "202.131.30.81"
- 16진수 0xcbf92703
- 이진값 11000000.1010...

바이트 순서

185 . 107 . 80 . 231

Big-endian : 0x B9 6B 50 E7

Little-endian : 0x E7 50 6B B9

	n번지	n+1번지	n+2번지	n+3번지
Big-endian	0x12	0x34	0x56	0x78
Little-endian	0x78	0x56	0x34	0x12

컴퓨터마다 어떤 방식을 사용하고 있을지 모른다 - > 네트워크 통신할 때는 통일을 해야한다.

네트워크 바이트 순서 (Network Byte Order) : Big-endian

호스트 바이트 순서 (Host Byte Order) : machine dependant (Big or Little)

n : network, h : host, s : short, l : long

ntohs()

ntohl()

htons()

htonl()

IP주소변환

202.131.30.81 -> 11001010 10000011 00011110 01010001

IP주소(dotted – decimal) -> IP주소(이진값, 32bit)

a : aski n : number

int inet_aton(const char *cp, struct in_addr *inp)

char* inet_nta(struct in_addr in)

```
struct in_addr
{
    in_addr_t      s_addr; // 32비트 IPv4 인터넷 주소
};
```

in_addr_t자료형 : uint32_t 이지만, in_addr_t로 씌으로써 코드가 무엇을 뜻하는지 알기 쉽게 하는 역할

Iterative echo program 서버 코드

```
11 int main(int argc, char *argv[])
12 {
13     int serv_sock, clnt_sock; // 파일 디스크립터 번호로 관리되기 때문에 int 자료
14     char message[BUF_SIZE];
15     int str_len, i=0;
16
17     struct sockaddr_in serv_adr;
18     struct sockaddr_in clnt_adr;
19     socklen_t clnt_adr_sz;
20
21     if(argc!=2) {
22         printf("Usage : %s <port>\n", argv[0]);
23         exit(1);
24     }
25
26     serv_sock=socket(PF_INET, SOCK_STREAM, 0);
27     if(serv_sock==-1)
28         error_handling("socket() error");
29
30     memset(&serv_adr, 0, sizeof(serv_adr));
31     serv_adr.sin_family=AF_INET;
32     serv_adr.sin_addr.s_addr=htonl(INADDR_ANY); // 모든 인터페이스 허용 0.0.0
33     serv_adr.sin_port=htons(atoi(argv[1])); // 포트번호 들어있는 곳
34
35     if(bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr))==-1)
36         error_handling("bind() error");
37
38     if(listen(serv_sock, 5)==-1)
39         error_handling("listen() error");
40
41     clnt_adr_sz=sizeof(clnt_adr);
42
43     while(1)
44     {
45         clnt_sock=accept(serv_sock, (struct sockaddr*)&clnt_adr, &clnt_adr_sz);
46         if(clnt_sock==-1)
47             error_handling("accept() error");
48         else
49             printf("Connected client %d \n", ++i);
50
51         printf("serv_sock PORT      : %d \n", ntohs(serv_adr.sin_port));
52         printf("client socket PORT : %d \n", ntohs(clnt_adr.sin_port));
53
54         struct sockaddr_in temp;
55         socklen_t len = sizeof(temp);
56
57         getsockname(clnt_sock, (struct sockaddr*)&temp, &len);
58
59         printf("clnt_sock PORT      : %d\n", ntohs(temp.sin_port));
60
61         while((str_len=read(clnt_sock, message, BUF_SIZE))!=0)
62             write(clnt_sock, message, str_len);
63
64         close(clnt_sock);
65     }
66
67     close(serv_sock);
68     return 0;
69 }
```

Iterative echo program 클라이언트 코드

```
11  int main(int argc, char *argv[])
12  {
13      int sock;
14      char message[BUF_SIZE];
15      int str_len;
16      struct sockaddr_in serv_adr;
17
18      if(argc!=3) {
19          printf("Usage : %s <IP> <port>\n", argv[0]);
20          exit(1);
21      }
22
23      sock=socket(PF_INET, SOCK_STREAM, 0);
24      if(sock==-1)
25          error_handling("socket() error");
26
27      memset(&serv_adr, 0, sizeof(serv_adr));
28      serv_adr.sin_family=AF_INET;
29      serv_adr.sin_addr.s_addr=inet_addr(argv[1]);
30      serv_adr.sin_port=htons(atoi(argv[2]));
31
32      if(connect(sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr))==-1)
33          error_handling("connect() error!");
34      else
35          puts("Connected.....");
36
37      while(1)
38      {
39          fputs("Input message(Q to quit): ", stdout);
40          fgets(message, BUF_SIZE, stdin);
41
42          if(!strcmp(message, "q\n") || !strcmp(message, "Q\n"))
43              break;
44
45          write(sock, message, strlen(message));
46          str_len=read(sock, message, BUF_SIZE-1);
47          message[str_len]=0;
48          printf("Message from server: %s", message);
49      }
50
51      close(sock);
52      return 0;
53  }
```

Iterative echo program

```
k@k:/mnt/c/Users/k/Desktop/SocketP/IterativeEcho$ ./eserver 9190
Connected client 1
serv_sock PORT      : 9190
client socket PORT  : 52596
clnt_sock PORT      : 9190
Connected client 2
serv_sock PORT      : 9190
client socket PORT  : 51100
clnt_sock PORT      : 9190
```

서버

```
k@k:/mnt/c/Users/k/Desktop/SocketP/IterativeEcho$ ./eclient 127.0.0.1 9190
Connected.....
Input message(Q to quit): q
k@k:/mnt/c/Users/k/Desktop/SocketP/IterativeEcho$ ./eclient 127.0.0.1 9190
Connected.....
Input message(Q to quit): hello
Message from server: hello
Input message(Q to quit): q
```

클라이언트

감사합니다